



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

13 April 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040

SYSTEM PROGRAMMING

WEEK 9 Lecture 2

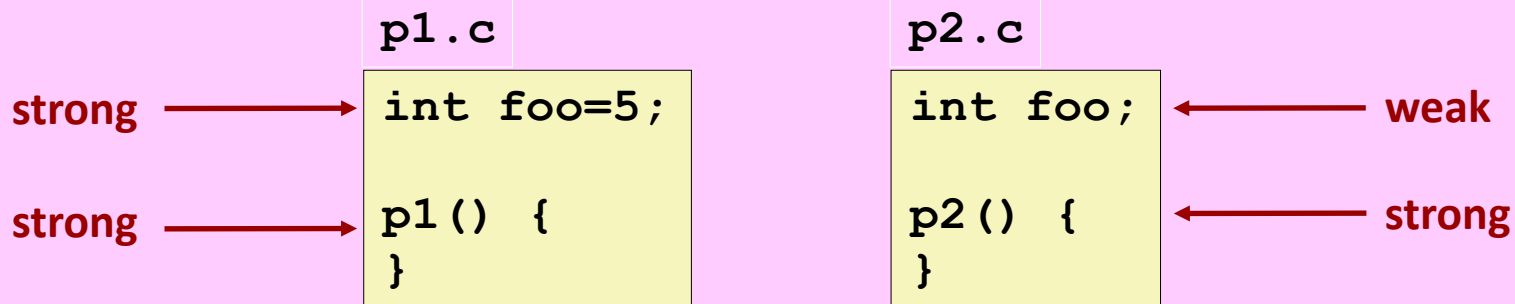
LINKING

PART 1 CONTINUED



How Linker Resolves Duplicate Symbol Definitions

- ★ Program symbols are either *strong* or *weak*
 - **Strong**: procedures and initialized globals
 - **Weak**: uninitialized globals



Linker's Symbol Rules

❑ Rule 1: Multiple strong symbols are not allowed

- Each item can be defined only once
- Otherwise: Linker error

```
int x;  
p1() {}
```

```
p1() {}
```

Link time error: two strong symbols (p1)

❑ Rule 2: Given a strong symbol and multiple weak symbols, choose the strong symbol

- References to the weak symbol resolve to the strong symbol

```
int x=7;  
p1() {}
```

```
int x;  
p2() {}
```

❑ Rule 3: If there are multiple weak symbols, pick an arbitrary one

```
int x;  
p1() {}
```

```
int x;  
p2() {}
```

Linker's Symbol Rules

QUESTION : List all the strong and weak symbols in the following code

```
/* swap.c */
```

```
extern int buf[];
```

Strong Symbols

Weak Symbols

```
int *bufp0 = &buf[0];
```

.....bufp0.....

..... buf

```
int *bufp1;
```

..... swap

..... bufp1.....

```
void swap() {
```

```
    int temp;
```

```
    bufp1 = &buf[1];
```

```
    temp = *bufp0;
```

```
    *bufp0 = *bufp1;
```

```
    *bufp1 = temp;
```

```
}
```

Global Variables

- ★ Avoid if you can
- ★ Otherwise
 - Use **static** if you can
 - Initialize if you define a global variable
 - Use **extern** if you reference an external global variable

SYMBOL TABLE

- ❑ Symbol tables are built by assemblers, using symbols exported by the compiler into the assembly-language .s file.
- ★ It contains an array of entries.

Format of each entry in the Symbol Table

typedef struct

{

int name; /*String Table Offset */

char type:4, /*Function or data (4 bits) */

binding:4; /*Local or Global (4bits) */

char reserved; /*unused*/

short section; /*Section header index */

long value; /*Section offset or absolute address */

long size; /*object size in bytes */

} ELF64_Symbol;

SYMBOL TABLE

❑ Symbol table Fields

typedef struct

{

int name; /*String Table Offset*/

The name is a byte offset into the string table that points to the null-terminated string name of the symbol.

char type:4; /*Function or data (4 bits)*/

The type is usually either data or function.

binding:4; /*Local or Global (4bits)*/

The binding field indicates whether the symbol is local or global.

char reserved; /*unused*/

short section; /*Section header index*/

Each symbol is associated with some section of the object file, denoted by the section field, which is an index into the section header table.

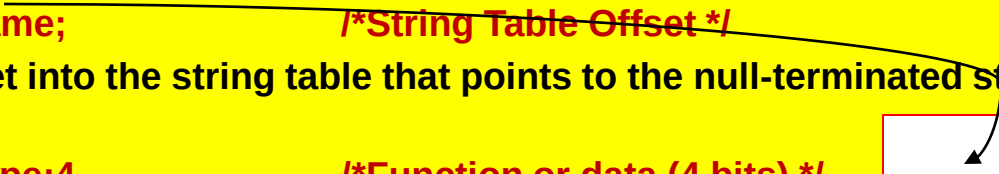
long value; /*Section offset or absolute address*/

- * The value is the symbol's address.
- * For relocatable modules, the value is an offset from the beginning of the section where the object is defined.
- * For executable object files, the value is an absolute run-time address.

long size; /*object size in bytes*/

The size is the size (in bytes) of the object.

} ELF64_Symbol;



byte offset	.strtab section
0	buf\0
4	bufp0\0
10	bufp1\0
16	main\0

SYMBOL TABLE

❑ SPECIAL PSEUDO SECTIONS

- ★ There are three special pseudo sections that don't have entries in the section header table:
- ★ **ABS** is for symbols that should not be relocated.
- ★ **UNDEF** is for undefined symbols, that is, symbols that are referenced in this object module but defined elsewhere.
- ★ **COMMON** is for uninitialized data objects that are not yet allocated.
- ★ For **COMMON** symbols, the value field gives the alignment requirement, and size gives the minimum size.
Example : **int a[100];** a is the uninitialized data object
value=4, i.e. alignment and size=400
→ symbol **a** belongs to pseudo section COM

SYMBOL TABLE

(a) main.c

```
/* main.c */
```

```
void swap();
```

```
int buf[2] = {1, 2};
```

```
int main()
```

```
{
```

```
    swap();
```

```
    return 0;
```

```
}
```

(b) swap.c

```
/* swap.c */
```

```
extern int buf[];
```

```
int *bufp0 = &buf[0];
```

```
int *bufp1;
```

```
void swap()
```

```
{
```

```
    int temp;
```

```
    bufp1 = &buf[1];
```

```
    temp = *bufp0;
```

```
    *bufp0 = *bufp1;
```

```
    *bufp1 = temp;
```

```
}
```

SYMBOL TABLE

- For example, here are the last three entries in the symbol table for main.o, as displayed by the **GNU readelf tool**.
- The first eight entries, which are not shown, are local symbols that the linker uses internally.

					Address	section		
Num:	Value	Size	Type	Bind	Ot	Ndx	Name	
8:	0	8	OBJECT	GLOBAL	0	3	buf	
9:	0	17	FUNC	GLOBAL	0	1	main	
10:	0	0	NOTYPE	GLOBAL	0	UND	swap	

In this example, we see an entry for the **definition** of global symbol **buf**, an 8 byte **object** located at an offset (i.e., value) of zero in the **.data** section.

This is followed by the definition of the **global** symbol **main**, a 17-byte function located at an offset of zero in the **.text** section.

The last entry comes from the **reference** for the external symbol **swap**.

Ndx=1 denotes the **.text** section, and
Ndx=3 denotes the **.data** section.

Read elf identifies each section by an integer index.

(a) main.c

```
/* main.c */  
void swap();  
int buf[2] = {1, 2};  
  
int main()  
{  
    swap();  
    return 0;  
}
```

SYMBOL TABLE

❑ Symbol table entries for swap.o:

				Address		section		
Num:	Value	Size	Type	Bind	Ot	Ndx	Name	
8:	0	4	OBJECT	GLOBAL	0	3	bufp0	→
9:	0	0	NOTYPE	GLOBAL	0	UND	buf	→
10:	0	39	FUNC	GLOBAL	0	1	swap	→
11:	4	4	OBJECT	GLOBAL	0	COM	bufp1	→

First, we see an entry for the definition of the **global** symbol **bufp0**, which is a 4 byte initialized object starting at offset 0 in **.data**.

The next symbol comes from the **reference** to the **external** **buf** symbol in the initialization code for **bufp0**.

This is followed by the **global** symbol **swap**, a 39-byte function at an offset of zero in **.text**.

The last entry is the **global** symbol **bufp1**, a 4-byte uninitialized data object (with a 4-byte alignment requirement) that will eventually be allocated as a **.bss** object when this module is linked.

(b) **swap.c**

```
/* swap.c */
```

```
extern int buf[];
```

```
int *bufp0 = &buf[0];
```

```
int *bufp1;
```

```
void swap()
```

```
{
```

```
    int temp;
```

```
    bufp1 = &buf[1];
```

```
    temp = *bufp0;
```

```
    *bufp0 = *bufp1;
```

```
    *bufp1 = temp;
```

```
}
```

SYMBOL TABLE

❑ Symbol table entries for swap.o:

				Address		section		
Num:	Value	Size	Type	Bind	Ot	Ndx	Name	
8:	0	4	OBJECT	GLOBAL	0	3	bufp0	→
9:	0	0	NOTYPE	GLOBAL	0	UND	buf	→
10:	0	39	FUNC	GLOBAL	0	1	swap	→
11:	4	4	OBJECT	GLOBAL	0	COM	bufp1	→

First, we see an entry for the definition of the **global** symbol **bufp0**, which is a 4 byte initialized object starting at offset 0 in **.data**.

The next symbol comes from the **reference** to the **external** **buf** symbol in the initialization code for **bufp0**.

This is followed by the **global** symbol **swap**, a 39-byte function at an offset of zero in **.text**.

The last entry is the **global** symbol **bufp1**, a 4-byte uninitialized data object (with a 4-byte alignment requirement) that will eventually be allocated as a **.bss** object when this module is linked.

(b) swap.c

```
/* swap.c */
```

```
extern int buf[];
```

```
int *bufp0 = &buf[0];
```

```
int *bufp1;
```

```
void swap()
```

```
{
```

```
    int temp;
```

```
    bufp1 = &buf[1];
```

```
    temp = *bufp0;
```

```
    *bufp0 = *bufp1;
```

```
    *bufp1 = temp;
```

```
}
```

Step 2: Relocation

□ Relocation

- ★ Once the linker has completed the symbol resolution step, it has associated each symbol reference in the code with exactly one symbol definition (i.e., a symbol table entry in one of its input object modules).
 - ★ At this point, the linker knows the exact sizes of the code and data sections in its input object modules.
 - ★ It is now ready to begin the relocation step, where it merges the input modules and assigns run-time addresses to each symbol.
- Relocation consists of two steps:
- **Relocating sections and symbol definitions**
 - **Relocating symbol references within sections**

Step 2: Relocation

STEP1 : Relocating sections and symbol definitions

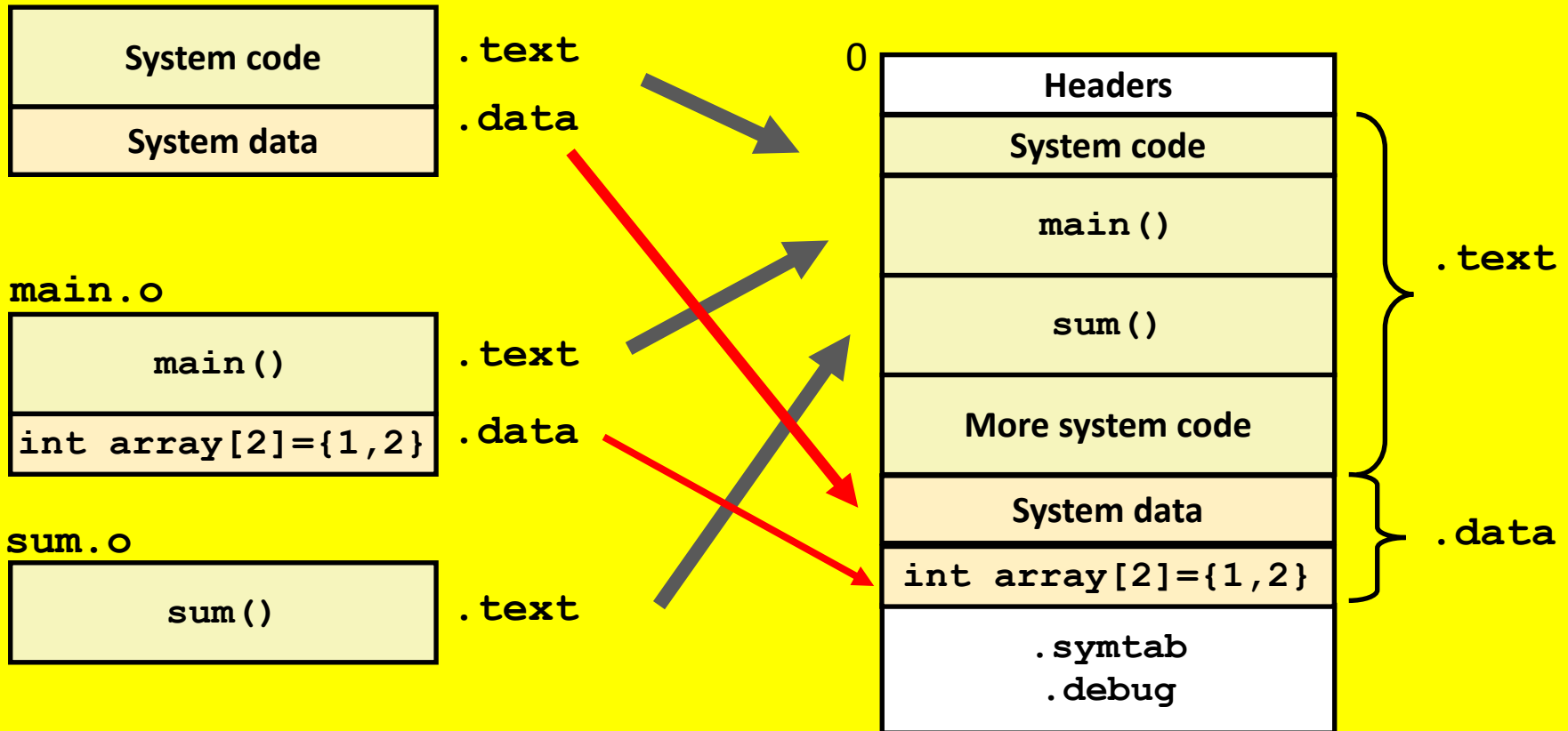
- ★ In this step, the linker merges all sections of the same type into a new aggregate section of the same type.
- ★ For example, the .data sections from the input modules are all merged into one section that will become the .data section for the output executable object file.
- ★ The linker then assigns run-time memory addresses to the new aggregate sections, to each section defined by the input modules, and to each symbol defined by the input modules.
- ★ When this step is complete, every instruction and global variable in the program has a unique run-time memory address.

Step 2: Relocation

STEP1 : Relocating sections and symbol definitions

Relocatable Object Files

Executable Object File



Step 2: Relocation

STEP2 :Relocating symbol references within sections

- ★ In this step, the linker modifies every symbol reference in the bodies of the code and data sections so that they point to the correct run-time addresses.
- ★ To perform this step, the linker relies on data structures in the relocatable object modules known as relocation entries.

Relocation Entries

- ★ When an assembler generates an object module, it does not know where the code and data will ultimately be stored in memory.
- ★ Nor does it know the locations of any externally defined functions or global variables that are referenced by the module.
- ★ So whenever the assembler encounters a reference to an object whose ultimate location is unknown, it generates a relocation entry that tells the linker how to modify the reference when it merges the object file into an executable.
- ★ Relocation entries for code are placed in **.rel.text**.
- ★ Relocation entries for initialized data are placed in **.rel.data**.

Relocation Entries

code/link/elfstructs.c

typedef struct

{

int offset; /* Offset of the reference to relocate */

int type:32; /* Relocation type */

symbol:32; /* Symbol Table Index */

long addend; /* Constant part of relocation expression */

} **Elf64_Rela;**

Figure: Format of ELF relocation entry

- ★ Each entry identifies a reference that must be relocated and specifies how to compute the modified reference.
- ★ The offset is the section offset of the reference that will need to be modified.
- ★ The symbol identifies the symbol that the modified reference should point to.
- ★ The type tells the linker how to modify the new reference.
- ★ The addend is a signed constant that is used by some types of relocations to bias the value of the modified reference.

Relocation Entries

- ★ ELF defines 32 different relocation types - two most basic relocation types are:

R_386_64_PC32

Relocate a reference that uses a 32-bit PC-relative address.

R_386_64_32

Relocate a reference that uses a 32-bit absolute address.

Relocation Entries

R_386_64_PC32: Relocate a reference that uses a 32-bit PC-relative address.

- ★ Recall that a PC-relative address is an offset from the current run-time value of the program counter(PC).
- ★ When the CPU executes an instruction e.g. call instruction using PC-relative addressing, it forms the effective address(e.g., the target address of the call instruction) by adding the 32-bit value i.e., offset encoded in the instruction to the current run-time value of the PC, which is always the address of the next instruction in memory.

Uses PC-Relative Addressing

- ★ **Effective Address(Next PC) = Current PC + 32-bit offset**

..... main:	
1FFB: e8 00 00 00 00	callq sum
2000: 48 63 ca	movslq %edx,%rcx
2003:	..
204F: c3	retq
2050: 48 83 c4 08	sum: add \$0x8,%rsp
.....	
2200: c3	retq

32-bit offset needs to be computed

When we execute the instruction callq sum at address 1FFB, PC gets updated to the address of the next instruction , i.e. PC becomes= 1FFB+5 = 2000 = Current PC, address of the next instruction. Target Function sum is located at address 2050

32-bit offset required is = Target sum address – Current PC = 2050 – 2000= 00 00 00 50
Since callq uses PC-Relative Addressing, Next PC = Current PC +32-bit offset = 2000 +50

main.c

Relocation Entries

```
int array[2] = {1, 2};

int main()
{
    int val = sum(array, 2);
    return val;
}
```

Since call instruction uses PC Relative addressing, **32-bit offset** needs to be computed as

Target address of sum – byte offset of the reference + addend

Here addend = -4 i.e., a correction factor because during the execution of call instruction, the PC will be updated to the address of the next instruction which is to be considered instead of byte offset of the reference

main.o

0000000000000000 <main>:

```
0: 48 83 ec 08      sub $0x8,%rsp
4: be 02 00 00 00   mov $0x2,%esi
9: bf 00 00 00 00   mov $0x0,%edi
e: e8 00 00 00 00   callq 13 <main+0x13>
13: 48 83 c4 08      add $0x8,%rsp
17: c3              retq
```

%edi = &array

call sum

Call instruction uses PC-Relative addressing, i.e., required relocation type = R_386_64_PC32

Function to be called here is sum, i.e., required Symbol = sum

Byte offset of the reference = f

f: R_386_64_PC32 sum -0x4

← Relocation entry required

Relocation Entries

R_386_64_32: Relocate a reference that uses a 32-bit absolute address.

- ★ With absolute addressing, the CPU directly uses the 32-bit value encoded in the instruction as the effective address, without further modifications.

Example showing absolute addressing

0000000000000000 <main>:

0: 48 83 ec 08 sub \$0x8,%rsp

4: be 02 00 00 00 mov \$0x2,%esi

9: **bf 00 00 00 00** **mov \$0x0,%edi**

e: e8 00 00 00 00 callq 13 <main+0x13>

13: 48 83 c4 08 add \$0x8,%rsp

17: c3 retq

Byte offset
of the
reference =
a

main.o

symbol= array

%edi = &array
call sum

Since mov instruction uses Absolute
addressing, required relocation type =
R_386_64_32

In Linking phase- assume that absolute address of array is known i.e. **\$0x601018** then the corresponding instruction machine code will be as follows:

bf 18 10 60 00 mov \$0x601018,%edi # %edi = &array

a: R_386_64_32 array

← **Relocation entry required**

addend=0

★ These two relocation types support the x86-64 small code model, which assumes that the total size of the code and data in the executable object file is smaller than 2GB , and thus can be accessed at run-time using 32-bit PC-relative addresses.

★ The small code model is the default for GCC. Programs larger than 2 GB can be compiled using the -mcmodel=medium (medium code model) and -mcmodel=large (large code model) flags.

Relocation Entries

```
int array[2] = {1, 2};
```

```
int main()
```

```
{  
    int val = sum(array, 2);  
    return val;  
}
```

main.c

Include Relocation entries

-d Diassemble Object
code to assembly
language Instructions

```
$objdump -r -d main.o
```

0000000000000000 <main>:

0: 48 83 ec 08 sub \$0x8,%rsp

4: be 02 00 00 00 mov \$0x2,%esi

9: bf 00 00 00 00 mov \$0x0,%edi

a: R_X86_64_32 array

Relocation entry

%edi = &array

e: e8 00 00 00 00 callq 13 <main+0x13>

f: R_X86_64_PC32 sum -0x4

Relocation entry

call sum()

13: 48 83 c4 08 add \$0x8,%rsp

17: c3 retq

main.o

Object Code

Assembly Code

13 April 2024

Lecture Slides on SOC 2040 SYSTEMS PROGRAMMING
SPRING Semester 2024 @ Dr A R Naseer

24

Relocating Symbol References

Figure : Relocation algorithm

line

```
1      foreach section s {
2          foreach relocation entry r {
3              refptr= s + r.offset ;    /* ptr to reference to be relocated */
4              /* Relocate a PC-relative reference */
5              if (r.type == R_X86_64_PC32)
6              {
7                  refaddr = ADDR(s) + r.offset;    /* ref's run-time address */
8                  *refptr = (unsigned) (ADDR(r.symbol) + r.addend - refaddr);
9              }
10             /* Relocate an absolute reference */
11             if (r.type == R_X86_64_32)
12                 *refptr = (unsigned) (ADDR(r.symbol) + r.addend);
13         }
    }
```

- * In the pseudo code for the linker's relocation algorithm. Lines 1 and 2 iterate over each section *s* and each relocation entry *r* associated with each section.
- * For concreteness, assume that each section *s* is an array of bytes and that each relocation entry *r* is a struct of type `Elf64_Rela`
- * Also, assume that when the algorithm runs, the linker has already chosen run-time addresses for each section (denoted `ADDR(s)`) and each symbol (denoted `ADDR(r.symbol)`).
- * Line 3 computes the address in the *s* array of the 4-byte reference that needs to be relocated. If this reference uses PC-relative addressing, then it is relocated by lines 5–9.
- * If the reference uses absolute addressing, then it is relocated by lines 11-12.

Relocating PC-Relative References



- * Consider the example in the previous slide in which the main routine in the .text section of **main.o** calls the **sum** routine, which is defined in **sum.o**.
- * The call instruction begins at section offset 0xe and consists of the 1-byte opcode 0xe8, followed by a placeholder for the 32-bit PC-relative reference to the target sum.
- * Here is the disassembled listing for the call instruction, as generated by the GNU objdump tool:

```
e: e8 00 00 00 00    callq 13 <main+0x13>           # sum()
      f: R_X86_64_PC32 sum -0x4    # Relocation entry
```

- * From this listing, we see that the call instruction begins at section offset 0xe and consists of the 1-byte opcode 0xe8, followed by the 32-bit reference 0x00000000 which is stored in little-endian byte order.
- * We also see a relocation entry for this reference displayed on the following line.
- * The corresponding relocation entry r consists of four fields:

r.offset = 0xf → f: R_X86_64_PC32 sum -0x4
r.type = R_X86_64_PC32 →
r.symbol = sum →
r.addend = - 4 →

```
0: 48 83 ec 08      sub  $0x8,%rsp
4: be 02 00 00 00    mov  $0x2,%esi
9: bf 00 00 00 00    mov  $0x0,%edi
      a: R_X86_64_32 array

e: e8 00 00 00 00    callq 13 <main+0x13> )
      f: R_X86_64_PC32 sum -0x4
13: 48 83 c4 08      add  $0x8,%rsp
17: c3               retq
```

These fields tell the linker to modify the 32-bit PC-relative reference starting at offset 0xf so that it will point to the sum routine at run time.

Relocating PC-Relative References

- Now, suppose that the linker has determined the starting run-time address for text $\text{ADDR}(s) = \text{ADDR}(\text{.text}) = 0x4004d0$ and $\text{ADDR}(r.\text{symbol}) = \text{ADDR}(\text{sum}) = 0x4004e8$

Using the relocation algorithm, the linker first computes the run-time address of the reference (line 7):

$$\text{refaddr} = \text{ADDR}(s) + r.\text{offset} = 0x4004d0 + 0xf = 0x4004df$$

- It updates the reference so that it will point to the sum routine at run time (line 8):
$$\begin{aligned} * \text{refptr} &= (\text{unsigned}) (\text{ADDR}(r.\text{symbol}) + r.\text{addend} - \text{refaddr}) \\ &= (\text{unsigned}) (0x4004e8 + (-4) - 0x4004df) = (\text{unsigned}) (0x5) \end{aligned}$$
- In the resulting executable object file, the call instruction has the following relocated form:

4004de: e8 05 00 00 00 callq 4004e8 <sum> sum();

- At run time, the call instruction will be stored at address 0x4004de. When the CPU executes the call instruction, the PC has a value of 0x4004e3, which is the address of the instruction immediately following the call instruction.

To execute the instruction, the CPU performs the following steps:

1. push PC onto stack

$$\text{2. PC} \leftarrow \text{PC} + 0x5 = 0x4004e3 + 0x5 = 0x4004e8$$

Thus, the next instruction to execute is the first instruction of the sum routine

```
4004d0: 48 83 ec 08      sub $0x8,%rsp
4004d4: be 02 00 00 00   mov $0x2,%esi
4004d9: bf 00 00 00 00   mov $0x0,%edi
                     a: R_X86_64_32 array
4004de: e8 00 00 00 00   callq 13 <main+0x13> )
                     f: R_X86_64_PC32 sum-0x4
4004e3: 48 83 c4 08      add $0x8,%rsp
4004e7: c3              retq
```

Relocating Absolute References

- * Relocating absolute references is straightforward. For example, in line 4 , the mov instruction copies the address of array (a 32-bit immediate value) into register %edi.
- * The mov instruction begins at section offset 0x9 and consists of the 1-byte opcode 0xbf, followed by a placeholder for the 32-bit absolute reference to array.

- * The corresponding relocation entry r consists

of four fields: **r.offset = 0xa**

r.type = R_X86_64_32

r.symbol = array

r.addend = 0

a: R_X86_64_32 array

```

4004d0: 48 83 ec 08      sub $0x8,%rsp
4004d4: be 02 00 00 00   mov $0x2,%esi
4004d9: bf 00 00 00 00   mov $0x0,%edi
                        a: R_X86_64_32 array
4004de: e8 00 00 00 00   callq 13 <main+0x13> )
                        f: R_X86_64_PC32 sum-0x4
4004e3: 48 83 c4 08      add $0x8,%rsp
4004e7: c3              retq
    
```

- * These fields tell the linker to modify the absolute reference starting at offset 0xa so that it will point to the first byte of **array** at run time.
- * Now, suppose that the linker has determined that the virtual memory address of array in the data section is **ADDR(r.symbol) = ADDR(array) = 0x601018**
- * The linker updates the reference using line 12 of the algorithm :

$$\begin{aligned}
 *refptr &= (\text{unsigned}) (\text{ADDR}(\text{r.symbol}) + \text{r.addend}) \\
 &= (\text{unsigned}) (0x601018 + 0) = (\text{unsigned}) (0x601018)
 \end{aligned}$$

- * In the resulting executable object file, the mov instruction has the following relocated form: **4004d9: bf 18 10 60 00 mov \$0x601018, %edi %edi= array**

- * At load time, the loader can copy the bytes from these sections directly into memory and execute the instructions without any further modifications

Relocated .text section

00000000004004d0 <main>:

```
4004d0: 48 83 ec 08          sub    $0x8,%rsp
4004d4: be 02 00 00 00      mov    $0x2,%esi
4004d9: bf 18 10 60 00      mov    $0x601018,%edi      # %edi = &array
4004de: e8 05 00 00 00      callq 4004e8 <sum>  # sum()
4004e3: 48 83 c4 08          add    $0x8,%rsp
4004e7: c3                  retq
```

00000000004004e8 <sum>:

```
4004e8: b8 00 00 00 00      mov    $0x0,%eax
4004ed: ba 00 00 00 00      mov    $0x0,%edx
4004f2: eb 09              jmp    4004fd <sum+0x15>
4004f4: 48 63 ca          movslq %edx,%rcx
4004f7: 03 04 8f          add    (%rdi,%rcx,4),%eax
4004fa: 83 c2 01          add    $0x1,%edx
4004fd: 39 f2            cmp    %esi,%edx
4004ff: 7c f3            jl     4004f4 <sum+0xc>
400501: f3 c3            rep retq
```

Using PC-relative addressing for sum(): $0x4004e8 = 0x4004e3 + 0x5$

Source: `objdump -dx prog`

Loading Executable Object Files

Executable Object File

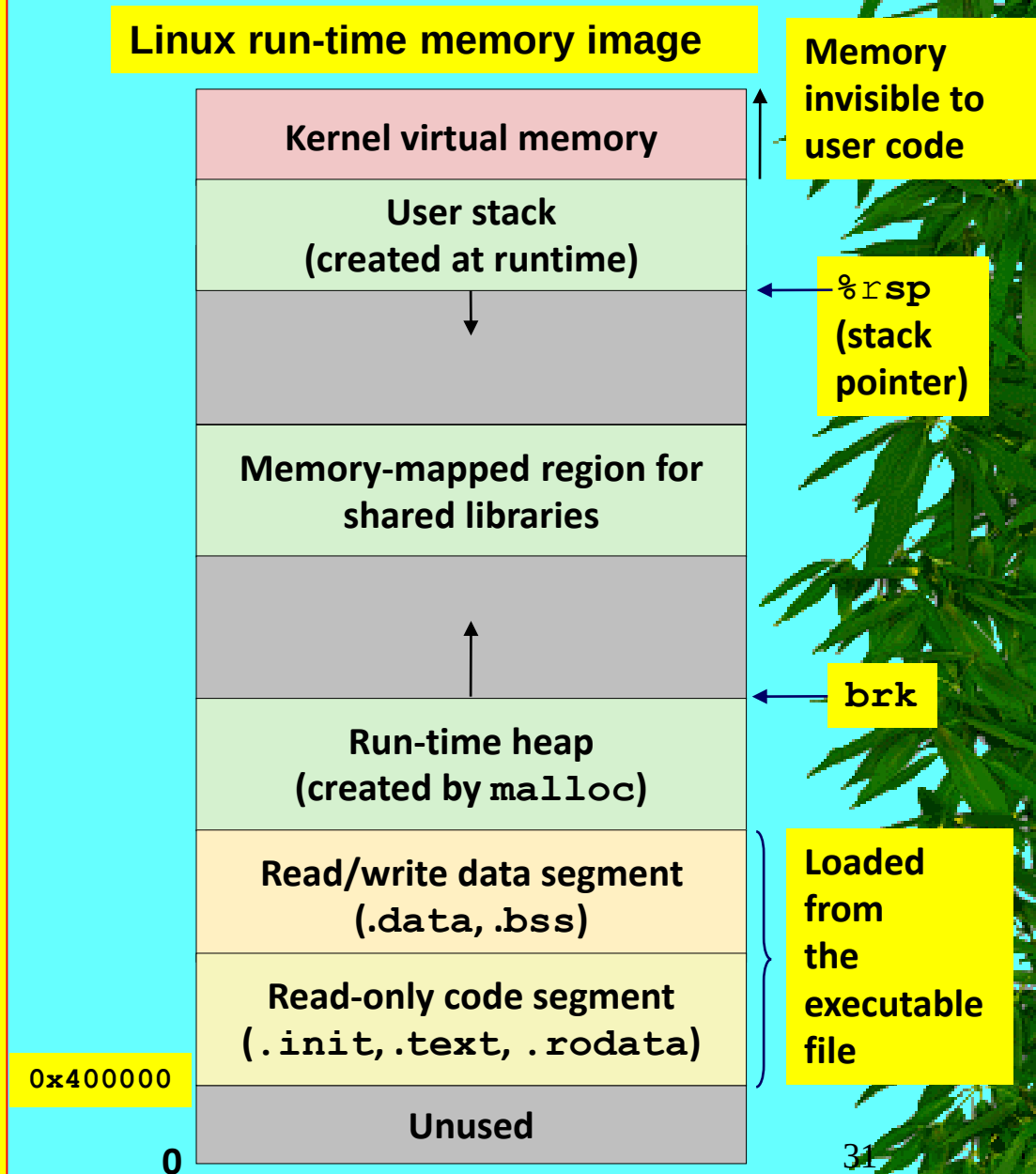
ELF header
Program header table (required for executables)
.init section
.text section
.rodata section
.data section
.bss section
.symtab
.debug
.line
.strtab
Section header table (required for relocatables)

0

- To run an executable object file `p`, we can type its name to the Linux/Unix shell's command line:
`$ ./p`
- Since `p` does not correspond to a built-in shell command, the shell assumes that `p` is an executable object file, which it runs for us by invoking some memory resident operating system code known as the **loader**.
- Any Linux/Unix program can invoke the loader by calling the **execve** function.
- The loader copies the code and data in the executable object file from disk into memory, and then runs the program by jumping to its first instruction, or entry point.
- This process of copying the program into memory and then running it is known as loading.

Loading Executable Object Files

- Every Linux/Unix program has a run-time memory image similar to the one in Figure.
- On 32-bit Linux systems, the code segment starts at address 0x08048000 and on 64-bit Linux systems, code segment at 0x400000
- The data segment follows at the next 4KB aligned address.
- The run-time heap follows on the first 4 KB aligned address past the read/write segment and grows up via calls to the malloc library.
- There is also a segment that is reserved for shared libraries.
- The user stack always starts at the largest legal user address and grows down (toward lower memory addresses).
- The segment starting above the stack is reserved for the code and data in the memory-resident part of the operating system known as the kernel.



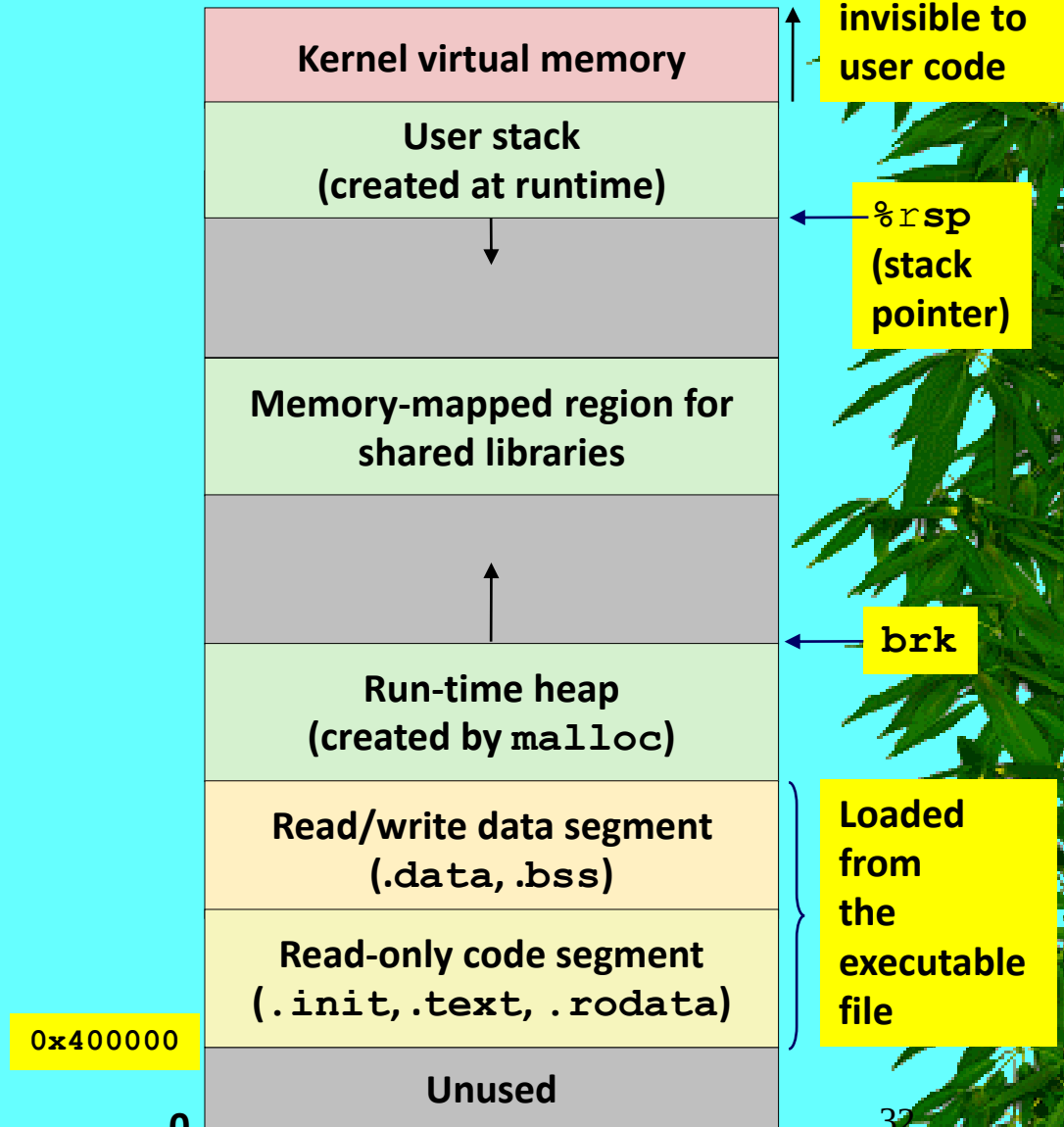
Loading Executable Object Files

Executable Object File

ELF header
Program header table (required for executables)
.init section
.text section
.rodata section
.data section
.bss section
.symtab
.debug
.line
.strtab
Section header table (required for relocatables)

0

Linux run-time memory image



SOC 2040
SYSTEM PROGRAMMING

END OF
WEEK 9 Lecture 2
LINKING PART 1

WISH YOU ALL THE BEST

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 9 of Text Book

➤ Computer Systems : A Programmer's Perspective

- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

13 April 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG